

Phase 3

Prerequisite: Phase 2 complete — all 11 verification items passed

The Prompt Dump File

Location and Naming

The prompt dump is a single fixed file. It is always written to the same path and overwritten on every new coding task. There is no versioning or timestamping — the file always represents the current task.

```
~/local_agent/prompt_dump.json
```

Note: Because this file is overwritten each run, completing one coding task before starting another is important. Phase 4 will read whatever is currently in this file — if a new task overwrites it before the coder runs, the previous task is lost.

JSON Structure

The prompt dump must follow this exact structure. Every field is required. No field may be null or empty — if a value is genuinely not applicable, use the string 'none'.

```
{
  "task_id": "<timestamp in YYYYMMDD_HHMMSS format>",
  "language": "<C# Console | C# WPF | C# MAUI | Python>",
  "task_type": "<scaffold | boilerplate | debug | assess>",
  "output_scope": "<method | class | module | fix>",
  "existing_code": "<none | paste of existing code as a single string>",
  "constraints": [
    "<constraint 1>",
    "<constraint 2>"
  ],
  "user_request": "<rewritten machine-readable instruction>",
  "status": "READY_FOR_CODER"
}
```

Warning: The status field must always be exactly the string `READY_FOR_CODER` — no spaces, no punctuation. Phase 4 will check this field before loading the coder model. If it is missing or malformed, the coder will not run.

Example Prompt Dump

Based on the WPF button handler example used in Phase 2 testing, a correctly formed dump looks like this:

```
{
  "task_id": "20260424_143512",
  "language": "C# WPF",
  "task_type": "scaffold",
  "output_scope": "method",
```

Phase 3

Prerequisite: Phase 2 complete — all 11 verification items passed

```
"existing_code": "none",
"constraints": [
  "Use MVVM pattern",
  "Implement ICommand interface",
  "No code-behind logic"
],
"user_request": "Scaffold a WPF button click handler using the MVVM
pattern with ICommand. Output a single method only. No code-behind.",
"status": "READY_FOR_CODER"
}
```

Note: *The sample dump here is for my purposes, a coder, so make a format that fits your purposes.*

Playbook Update — Dump Signal

The Phase 2 playbook already instructs the orchestrator to produce a structured task summary. One small addition is needed: the orchestrator must signal to the Python script that the task summary is complete and ready to be written to disk.

Open `playbook.md` and find the end of the Phase 2 coding task protocol. Replace the STATUS line:

Find this line (Phase 2 version):

```
STATUS: READY FOR DELEGATION — awaiting Phase 3 implementation.
```

Replace it with (Phase 3 version):

```
STATUS: READY_FOR_CODER
```

Also add this instruction directly above the STATUS line in the task summary template:

```
After producing this summary, output the single line:
DUMP_READY
on its own line, after the summary block. This signals the system to
write the dump file.
```

Key Point: *DUMP_READY is the signal the Python script watches for. When main.py sees this token in the orchestrator's response, it extracts the task summary, converts it to JSON, and writes the dump file. Without this signal, the script cannot know when the summary is complete.*

Save and exit: Ctrl+O, Enter, Ctrl+X.

Phase 3

Prerequisite: Phase 2 complete — all 11 verification items passed

New File — dump_writer.py

This is the only new Python file in Phase 3. It takes the orchestrator's plain text task summary, parses it into a Python dictionary, and writes it as a valid JSON file to the configured dump path.

```
nano ~/local_agent/dump_writer.py
```

Paste the following exactly:

```
import json
import os
import re
from datetime import datetime
from config import PROMPT_DUMP_PATH

def parse_task_summary(text: str) -> dict:
    """
    Parse the orchestrator's plain text task summary into a dictionary.
    Expects the summary block produced by the Phase 3 playbook protocol.
    """
    def extract(label, fallback="none"):
        pattern = rf"{label}\s*:\s*(.*)"
        match = re.search(pattern, text, re.IGNORECASE)
        return match.group(1).strip() if match else fallback

    # Extract constraints — may be a comma-separated list
    raw_constraints = extract("Constraints", "")
    if raw_constraints and raw_constraints.lower() != "none":
        constraints = [c.strip() for c in raw_constraints.split(",") if c.strip()]
    else:
        constraints = []

    return {
        "task_id": datetime.now().strftime("%Y%m%d_%H%M%S"),
        "language": extract("Language/Ecosystem"),
        "task_type": extract("Task Type"),
        "output_scope": extract("Output Scope"),
        "existing_code": extract("Existing Code"),
        "constraints": constraints,
        "user_request": extract("User Request"),
        "status": "READY_FOR_CODER"
    }

def write_dump(task_summary_text: str) -> bool:
    """
    Parse the task summary and write it to the prompt dump JSON file.
    """
```

Phase 3

Prerequisite: Phase 2 complete — all 11 verification items passed

```
    Returns True on success, False on failure.
    """
    try:
        data = parse_task_summary(task_summary_text)

        # Validate required fields are not empty
        required = ["language", "task_type", "output_scope",
"user_request"]
        missing = [k for k in required if data[k] == "none" or not
data[k]]
        if missing:
            print(f"[DUMP ERROR] Missing required fields: {missing}")
            return False

        with open(PROMPT_DUMP_PATH, "w", encoding="utf-8") as f:
            json.dump(data, f, indent=2)

        print(f"[DUMP WRITTEN] {PROMPT_DUMP_PATH}")
        return True

    except Exception as e:
        print(f"[DUMP ERROR] Failed to write dump: {e}")
        return False

def read_dump() -> dict | None:
    """
    Read and return the current prompt dump as a dictionary.
    Returns None if the file does not exist or is malformed.
    """
    if not os.path.exists(PROMPT_DUMP_PATH):
        return None
    try:
        with open(PROMPT_DUMP_PATH, "r", encoding="utf-8") as f:
            return json.load(f)
    except Exception as e:
        print(f"[DUMP ERROR] Failed to read dump: {e}")
        return None
```

Save and exit: Ctrl+O, Enter, Ctrl+X.

Phase 3

Prerequisite: Phase 2 complete — all 11 verification items passed

Update config.py — Add Dump Path

Add the prompt dump file path to config.py so all scripts reference the same location:

```
nano ~/local_agent/config.py
```

Add the following line after the existing memory path definitions:

```
# Prompt dump path — orchestrator writes here, coder reads from here
PROMPT_DUMP_PATH = os.path.join(os.path.expanduser("~/local_agent"),
"prompt_dump.json")
```

Save and exit. The full config.py should now look like this:

```
import os

# Model configuration
ORCHESTRATOR_MODEL = "llama3.1:8b"

# Memory file paths
MEMORY_DIR          = os.path.expanduser("~/agent_memory")
USER_PROFILE_PATH    = os.path.join(MEMORY_DIR, "user_profile.md")
PLAYBOOK_PATH       = os.path.join(MEMORY_DIR, "playbook.md")
SESSION_NOTES_PATH   = os.path.join(MEMORY_DIR, "session_notes.md")
PROJECTS_PATH        = os.path.join(MEMORY_DIR, "projects.md")

# Prompt dump path — orchestrator writes here, coder reads from here
PROMPT_DUMP_PATH     = os.path.join(os.path.expanduser("~/local_agent"),
"prompt_dump.json")

# Ollama API
OLLAMA_HOST = "http://localhost:11434"
```

Phase 3

Prerequisite: Phase 2 complete — all 11 verification items passed

Update main.py — Detect DUMP_READY and Write File

main.py needs two small additions: import the dump writer, and check each orchestrator response for the DUMP_READY signal. When detected, extract the task summary block and call write_dump().

```
nano ~/local_agent/main.py
```

Replace the entire file with the following updated version:

```
import sys
import os
sys.path.insert(0, os.path.dirname(__file__))

from memory import load_memory
from ollama_client import chat
from dump_writer import write_dump

DUMP_SIGNAL = "DUMP_READY"

def build_system_prompt(memory: str) -> str:
    return (
        "You are a local teaching assistant agent.\n\n"
        + memory
    )

def extract_summary_block(response: str) -> str:
    """
    Extract the task summary block from the orchestrator response.
    Looks for content between 'CODING TASK DETECTED' and 'DUMP_READY'.
    """
    start = response.find("CODING TASK DETECTED")
    end = response.find(DUMP_SIGNAL)
    if start == -1 or end == -1:
        return response # fallback: pass full response to parser
    return response[start:end].strip()

def main():
    print("Loading memory...")
    memory = load_memory()
    system_prompt = build_system_prompt(memory)

    messages = [{"role": "system", "content": system_prompt}]

    print("Agent ready. Type your message. Press Ctrl+C to exit.\n")
```

Phase 3

Prerequisite: Phase 2 complete — all 11 verification items passed

```
while True:
    try:
        user_input = input("You: ").strip()
    except (KeyboardInterrupt, EOFError):
        print("\nSession ended.")
        break

    if not user_input:
        continue

    messages.append({"role": "user", "content": user_input})

    print("Agent: ", end="", flush=True)
    response = chat(messages)
    print(response)
    print()

    messages.append({"role": "assistant", "content": response})

    # Check for dump signal
    if DUMP_SIGNAL in response:
        summary_block = extract_summary_block(response)
        write_dump(summary_block)

if __name__ == "__main__":
    main()
```

Save and exit: Ctrl+O, Enter, Ctrl+X.

Phase 3

Prerequisite: Phase 2 complete — all 11 verification items passed

Updated Project Folder Structure

After Phase 3, the project folder contains one new file and one new runtime artifact:

Path	Status	Purpose
~/local_agent/main.py	Updated	Now watches for DUMP_READY and calls dump writer
~/local_agent/config.py	Updated	PROMPT_DUMP_PATH added
~/local_agent/dump_writer.py	New	Parses task summary, writes and reads JSON dump
~/local_agent/memory.py	Unchanged	Loads Level 1 and Level 2 memory files
~/local_agent/ollama_client.py	Unchanged	Ollama API wrapper
~/local_agent/prompt_dump.json	Created at runtime	The prompt dump — written after each flagged coding task
~/agent_memory/playbook.md	Updated	DUMP_READY signal instruction added

Note: *prompt_dump.json* does not exist until the first coding task is flagged and *DUMP_READY* is detected. This is expected — do not create it manually.

Phase 3

Prerequisite: Phase 2 complete — all 11 verification items passed

Phase 3 Test Protocol

Run all tests in order. Start the agent fresh before testing.

```
cd ~/local_agent && python3 main.py
```

STEP T1 dump_writer.py Imports Correctly

Confirm the new module loads without errors before running the agent.

Run this before starting the agent:

```
python3 -c "from dump_writer import write_dump, read_dump;
print('dump_writer OK')"
```

Pass: Prints 'dump_writer OK' with no errors.

Fail: ImportError or AttributeError — check that config.py has PROMPT_DUMP_PATH defined and that dump_writer.py was saved correctly.

STEP T2 DUMP_READY Signal Detected

Confirm main.py detects the signal in the orchestrator response.

Walk through a complete coding task in the agent session:

- Start the agent: python3 main.py
- Request a coding task and answer all clarifying questions
- Wait for the orchestrator to produce the task summary ending with DUMP_READY

Pass: The terminal prints [DUMP WRITTEN] ~/local_agent/prompt_dump.json immediately after DUMP_READY appears in the response.

Fail: [DUMP WRITTEN] does not appear. Either the orchestrator is not outputting DUMP_READY (check playbook update), or main.py is not detecting it (check the DUMP_SIGNAL constant in main.py).

STEP T3 Prompt Dump File Exists and Is Valid JSON

Confirm the file was written and contains parseable JSON.

After T2 passes, run:

```
cat ~/local_agent/prompt_dump.json
```

Pass: File contents display as valid JSON with all eight fields present: task_id, language, task_type, output_scope, existing_code, constraints, user_request, status.

Phase 3

Prerequisite: Phase 2 complete — all 11 verification items passed

Fail: File is empty, contains malformed JSON, or is missing fields. Check the `parse_task_summary` function in `dump_writer.py` — the regex patterns must match the exact field labels in the playbook summary template.

Validate JSON syntax formally:

```
python3 -c "import json; f=open('prompt_dump.json'); data=json.load(f);
print('Valid JSON'); print(data)"
```

Pass: Prints 'Valid JSON' followed by the dictionary contents.

Fail: `JSONDecodeError` — the `dump_writer` produced malformed output. Check for special characters in the `user_request` field that may be breaking JSON serialization.

STEP T4 All Required Fields Are Populated

Confirm no required field contains 'none' when it should have a real value.

Run:

```
python3 -c "
import json
with open('prompt_dump.json') as f: d = json.load(f)
required = ['language', 'task_type', 'output_scope', 'user_request']
for k in required:
    status = 'OK' if d.get(k) and d[k] != 'none' else 'MISSING'
    print(f'{k}: {status}')
"
```

Pass: All four required fields print OK.

Fail: Any field prints MISSING — the orchestrator did not gather that context before flagging. Return to Phase 2 testing and ensure T3 (sequential clarification) passes before retrying Phase 3.

STEP T5 Status Field Is Correct

Confirm the status field contains exactly READY_FOR_CODER.

```
python3 -c "import json; f=open('prompt_dump.json'); d=json.load(f);
print(d['status'])"
```

Pass: Prints exactly: `READY_FOR_CODER`

Fail: Prints anything else — check the `write_dump` function in `dump_writer.py`. The status field is hardcoded and should not vary.

Phase 3

Prerequisite: Phase 2 complete — all 11 verification items passed

STEP T6 File Is Overwritten on Second Task

Confirm fixed-location behaviour — old task is replaced by new task.

Run a second complete coding task in the same or a new session. Then:

```
cat ~/local_agent/prompt_dump.json
```

Pass: The file contains the second task's details, not the first. The task_id timestamp should be different from the first run.

Fail: File still contains the first task — write_dump is not overwriting. Check that the file open mode is 'w' (write), not 'a' (append).

STEP T7 Phase 1 and Phase 2 Regression

Confirm earlier behaviour is unaffected by Phase 3 changes.

- Memory still loads: ask 'What do you know about me?' — orchestrator responds with profile information
- Clarification still works: send a vague coding request — orchestrator asks one question at a time
- Non-coding tasks handled directly: ask an explanation question — no DUMP_READY signal appears, no file is written

Pass: All three behaviours work as in Phases 1 and 2.

Fail: Any regression — check that main.py changes did not accidentally remove the memory loading or conversation loop logic.

Troubleshooting

DUMP_READY signal not appearing in orchestrator response

- Confirm the playbook update was saved: `cat ~/agent_memory/playbook.md` — scroll to the bottom and verify the DUMP_READY instruction is present.
- Restart the agent — the orchestrator loads memory at startup only.
- If the model is inconsistently outputting DUMP_READY, strengthen the playbook instruction:
 - 'After producing the task summary, you MUST output the single word DUMP_READY on its own line. This is mandatory and must not be omitted.'

[DUMP WRITTEN] does not appear after DUMP_READY

- Check that main.py was saved correctly — run: `cat ~/local_agent/main.py` and confirm the DUMP_SIGNAL check block is present at the bottom of the while loop.
- Confirm dump_writer.py is in the same directory as main.py: `ls ~/local_agent/`
- Run the import test: `python3 -c "from dump_writer import write_dump; print('OK')"`

JSON file is written but fields contain 'none'

- The regex in `parse_task_summary` is not matching the field labels the orchestrator produced.
- Print the raw orchestrator response for a flagged task and compare the field label names exactly against the regex patterns in `dump_writer.py`.
- Common mismatch: the playbook template uses 'Language/Ecosystem' but the regex searches for 'Language' only — extend the pattern to match both.
- Fix: update the `extract()` call in `parse_task_summary` to use the exact label text the orchestrator outputs.

JSONDecodeError when reading the dump

- Special characters in the `user_request` field (quotes, backslashes, newlines) can break JSON serialization if they are not escaped.
 - The `json.dump()` call in `write_dump` handles this automatically — if you are seeing this error, check that you are not constructing the JSON manually anywhere.
 - If the problem persists, add this line before `json.dump` to sanitize the `user_request` field:
 - `data['user_request'] = data['user_request'].replace('"', '')`
-

Phase 3

Prerequisite: Phase 2 complete — all 11 verification items passed

What Phase 4 Will Build On

Phase 3 ends with a valid, well-formed JSON file sitting at `~/local_agent/prompt_dump.json`. Phase 4 reads that file and uses it as the sole input to the coder model.

Phase	Input	Output
Phase 3 (this document)	Orchestrator task summary in terminal	prompt_dump.json written to disk
Phase 4	prompt_dump.json	Code output written to <code>~/local_agent/coder_output.txt</code>
Phase 5	coder_output.txt	Orchestrator reads output and delivers summary to user — full pipeline complete

Phase 4 introduces the first invocation of Qwen2.5-Coder 7B. It will be a separate script — `coder.py` — that reads the dump file, builds a prompt from it, calls the coder model, and writes the result to disk. The orchestrator is not involved in Phase 4 execution — it has already done its job by writing the dump.

Tip: Before starting Phase 4, run three complete coding tasks through Phase 3 and inspect the dump files each time. The quality and consistency of the dump directly determines the quality of the coder's output in Phase 4. Any formatting issues in the dump are much easier to fix now than after the coder is wired in.